Universidad de San Carlos de Guatemala
Centro Universitario de Occidente "CUNOC"
División Ciencias de la Ingeniería
Introducción a la Programación y Computación IPC 2
Ing. Moisés Granados Guevara



Manual Técnico
Jasper International School

William Eduardo Gómez Sánchez
Carné: 202232064

Guatemala, Quetzaltenango, 30 de octubre de 2026

Introducción

El presente manual técnico documenta el diseño e implementación del Sistema de Gestión Escolar de Jasper International School, una aplicación web desarrollada para modernizar la administración de un colegio que abarca desde pre-primaria hasta diversificado. El sistema centraliza la gestión de usuarios y personal, la estructura académica (años lectivos, grados, carreras, cursos, secciones y asignación de maestros), las inscripciones, los pagos y solvencias, la biblioteca, el registro de asistencia y notas de los estudiantes, y la generación de reportes en PDF.

El sistema está construido bajo una arquitectura cliente-servidor desacoplada. El backend fue desarrollado en Java, expone una API REST implementada con Jersey sobre Jakarta EE y se despliega en Apache Tomcat; la persistencia se realiza en una base de datos relacional MySQL mediante JDBC con un pool de conexiones. La seguridad se gestiona con autenticación basada en tokens JWT, almacenamiento de contraseñas con BCrypt y control de acceso por roles. Los reportes se generan en formato PDF con JasperReports. El frontend fue desarrollado con Angular y Tailwind CSS, y consume la API del backend.

Este documento está dirigido a desarrolladores, personal técnico o cualquier persona interesada en comprender la arquitectura, el diseño de clases, la estructura de la base de datos y el funcionamiento interno del sistema. Se detallan los requisitos para su instalación y ejecución, las tecnologías y patrones de diseño empleados, la descripción de las clases del sistema organizadas por paquete, los diagramas de análisis y diseño correspondientes, así como el pseudocódigo de los procesos más relevantes de la aplicación.

Requisitos del Sistema

Requisitos mínimos para ejecutar el proyecto

- Sistema operativo Windows 10 o superior
- Java Development Kit (JDK) versión 17 (desarrollado con 17.0.12)
- Apache Tomcat 11.0.26 o compatible con Jakarta EE 11
- MySQL Server 8.0 o superior
- MySQL Workbench 8.0 (opcional, recomendado para administrar la base de datos)
- Node.js 24 y npm, para ejecutar el frontend
- Navegador web actualizado (Google Chrome, Microsoft Edge o similar)
- Mínimo 4 GB de memoria RAM disponible
- Carpeta `C:\jasper-imagenes\libros`, donde el servidor almacena las portadas de los libros. Debe existir y tener permisos de escritura para el usuario que ejecuta Tomcat.

Requisitos para compilar el proyecto

- Apache NetBeans (última versión estable) con Apache Maven integrado, para el backend
- Angular 22 y Angular CLI, para el frontend
- Jaspersoft Studio 6.21.5 (opcional), solo para editar las plantillas de reportes (`.jrxml`)
- Las dependencias del backend, declaradas en el archivo `pom.xml`:
  - `jakarta.jakartaee-api` 11.0.0-M1
  - `jakarta.ws.rs-api` 3.1.0
  - Jersey 4.0.0-M4 (`jersey-container-servlet`, `jersey-hk2`, `jersey-media-json-jackson`, `jersey-media-multipart`)
  - `jackson-datatype-jsr310` 2.18.2
  - `tomcat-jdbc` 11.0.11 (pool de conexiones)
  - `mysql-connector-j` 9.7.0
  - `bcrypt` (at.favre.lib) 0.10.2
  - `jjwt` (api, impl y jackson) 0.12.7
  - `commons-lang3` 3.18.0
  - `jasperreports` 6.21.3
  - `batik-bridge` 1.17

Preparar la base de datos

1. Abrir MySQL Workbench y conectarse al servidor local.
2. Ejecutar el script `mapeo_sistema_colegio.sql`, que crea la base `sistema_colegio` con todas sus tablas.
3. Ejecutar el script `datos_iniciales.sql`, que carga datos de prueba de todos los roles. La contraseña de todas las cuentas de prueba es `Colegio2026`.
4. Configurar la conexión a MySQL en el archivo `context.xml` de la aplicación, dentro de la etiqueta `<Resource auth="Container" driverClassName="com.mysql.cj.jdbc.Driver" factory="org.apache.tomcat.jdbc.pool.DataSourceFactory" initialSize="5" maxActive="20" maxIdle="10" maxWait="10000" minIdle="5" name="jdbc/colegio" password="" testOnBorrow="true" type="javax.sql.DataSource" url="jdbc:mysql://localhost:3306/sistema_colegio" username="gs_william237" validationQuery="SELECT 1"/>` del pool de conexiones: URL de la base (`jdbc:mysql://localhost:3306/sistema_colegio`), usuario y contraseña de MySQL del equipo donde se ejecuta.

Compilar y ejecutar el backend desde NetBeans

1. Abrir el proyecto con la opción Open Project.
2. Verificar que Apache Tomcat esté configurado en NetBeans (Tools → Servers).
3. Clic derecho sobre el proyecto y seleccionar Clean and Build, para compilar y descargar las dependencias.
4. Clic derecho y seleccionar Run, lo cual despliega la aplicación en Tomcat, que atiende en el puerto 8080.

Compilar el backend desde consola (Maven)

1. Abrir una terminal en la carpeta raíz del proyecto (donde está el `pom.xml`).
2. Ejecutar `mvn clean package`, que genera el archivo `.war` en la carpeta `target/`.
3. Copiar el `.war` a la carpeta `webapps` de Tomcat e iniciarlo con `startup.bat` (ubicado en `bin`).

Ejecutar el frontend

1. Abrir una terminal en la carpeta raíz del proyecto Angular.
2. Ejecutar `npm install` la primera vez, para descargar las dependencias.
3. Ejecutar `ng serve` (o `npm start`).
4. Abrir en el navegador `http://localhost:4200`. Es la dirección de acceso al sistema.

Descripción de Tecnologías

Java 17
Lenguaje de programación principal del backend, utilizado para la lógica de negocio, los controladores REST y el acceso a datos.

Jakarta EE 11 y Jersey (JAX-RS)
Jakarta EE provee las especificaciones del servidor. Jersey, la implementación de referencia de JAX-RS, se utiliza para construir la API REST: expone los recursos como rutas HTTP y convierte las peticiones y respuestas a JSON. La inyección de dependencias se resuelve con HK2.

Jackson
Biblioteca de serialización JSON utilizada por Jersey. Con su módulo `jsr310` se manejan correctamente los tipos de fecha y hora de Java.

Apache Tomcat 11.0.26
Contenedor de servlets donde se despliega el backend. Atiende las peticiones HTTP y aloja el pool de conexiones a la base de datos.

MySQL 8.0 y JDBC
MySQL es el sistema gestor de base de datos relacional donde se almacena toda la información. El backend se comunica con él mediante JDBC y el conector `mysql-connector-j`, usando sentencias preparadas (`PreparedStatement`) para evitar la inyección SQL.

Pool de conexiones (tomcat-jdbc)
Mantiene un conjunto de conexiones abiertas a la base de datos, que se reutilizan entre peticiones para mejorar el rendimiento.

JWT (JSON Web Tokens, jjwt)
Mecanismo de autenticación sin estado: al iniciar sesión, el servidor emite un token firmado que el frontend envía en cada petición. El backend lo valida y obtiene de él el usuario y su rol.

BCrypt
Algoritmo de hash utilizado para almacenar las contraseñas, de modo que nunca se guardan en texto plano.

JasperReports 6.21 y Jaspersoft Studio
Motor de generación de reportes. Las plantillas `.jrxml` se diseñan en Jaspersoft Studio y se compilan en el servidor para exportar los reportes en PDF. Se complementa con Apache Batik para el manejo de imágenes vectoriales.

Apache Maven
Herramienta de gestión de dependencias y compilación del backend.

Angular 22 y TypeScript
Framework utilizado para el frontend, una aplicación de página única (SPA). Se emplean rutas protegidas por guards según el rol, interceptors para adjuntar el token en cada petición y signals para el manejo del estado de la interfaz.

Tailwind CSS 4 y Bootstrap Icons
Tailwind se utiliza para el diseño de la interfaz, incluyendo el color distintivo de cada rol. Bootstrap Icons aporta la iconografía de los menús y las acciones.

Node.js 24 y npm
Entorno de ejecución y gestor de paquetes necesarios para compilar y ejecutar el frontend.

Git y GitHub
Control de versiones del proyecto, con ramas por iteración y etiquetas por entrega.

NetBeans. VisualCode y MySQL Workbench
Entornos usados para el desarrollo del backend, frontend y la administración de la base de datos.

Conceptos de Programación Orientada a Objetos Aplicados

Herencia
Las personas que usan el sistema se modelan con una clase base `Usuario`, que concentra los datos comunes (CUI, nombre, correo, teléfono, dirección y contraseña). De ella se especializan los distintos roles (`SuperAdmin`, `Admin`, `Secretaria`, `Bibliotecario`, `Maestro` y `Estudiante`), cada uno con sus propios atributos. Esta jerarquía se refleja en la base de datos con la estrategia de tabla por subclase: la tabla `usuario` guarda lo común y cada rol tiene su tabla, relacionada por clave primaria compartida.

Interfaces y polimorfismo
La capa de persistencia se define mediante interfaces. Existe una interfaz genérica `DAOGenerico`, y una interfaz por entidad (por ejemplo, `PagoDAO` o `LibroDAO`) que declara las operaciones permitidas; cada una cuenta con su clase de implementación (`PagoDAOPersistencia`, `LibroDAOPersistencia`). Los controladores dependen de la interfaz y no de la implementación, por lo que la forma de acceso a datos puede cambiar sin modificar la lógica de los controladores.

El mismo principio se aplica al envío de correos: la interfaz `EnviadorCorreo` define el contrato y `EnviadorCorreoConsola` es la implementación actual, que muestra el mensaje en la consola del servidor. Puede sustituirse por una implementación que use un servidor SMTP sin cambiar el resto del sistema.

Encapsulamiento
Los atributos de las clases del modelo se declaran privados y se acceden mediante métodos públicos, de modo que los datos se manipulan únicamente por las vías previstas.

Enumeraciones
Los estados y conceptos con valores fijos se modelan con enumeraciones (`EstadoPago`, `EstadoInscripcion`, `ConceptoPago`, `ConceptoTarifa`, `Nivel`, `Puesto`, entre otras), lo que evita valores inválidos y hace el código más legible.

Objetos de transferencia de datos (DTO)
Las entidades del modelo no se exponen directamente por la API. Se utilizan clases de solicitud (`Solicitud...`) para recibir datos del frontend y clases de respuesta (`Respuesta...`) para devolverlos, lo que permite controlar exactamente qué información sale del sistema (por ejemplo, nunca se envía la contraseña).

Manejo de excepciones personalizadas
El sistema define 34 excepciones propias que representan cada situación de negocio (`ExcepcionCorreoDuplicado`, `ExcepcionUltimoSuperAdmin`, `ExcepcionAsignacionAjena`, `ExcepcionSolapamientoFechas`, entre otras). Los controladores las convierten en respuestas HTTP con un mensaje claro para el usuario.

Arquitectura del Proyecto Backend

Arquitectura cliente-servidor en capas

El sistema se compone de dos aplicaciones independientes: un frontend en Angular, que se ejecuta en `http://localhost:4200`, y un backend que expone una API REST. Se comunican por HTTP con mensajes JSON, y cada petición viaja con un token JWT.

- El recorrido de una petición es el siguiente:
- El usuario interactúa con una pantalla de Angular. Un interceptor adjunta el token JWT a la petición.
- El filtro de autenticación del backend valida el token y verifica que el rol del usuario esté permitido para esa ruta.
- El controlador REST correspondiente recibe la solicitud, valida los datos y llama a la capa de persistencia.
- La capa de persistencia ejecuta las consultas SQL sobre MySQL y construye objetos del modelo.
- El controlador convierte el resultado en una respuesta, ya sea un objeto JSON o un PDF, y la devuelve al frontend, que actualiza la pantalla.

Estructura de paquetes del backend

El backend se organiza en los siguientes paquetes, dentro de `com.usac.colegio.jasperschoolbackend`:

controlador: 24 clases que exponen la API REST, una por módulo (autenticación, usuarios, académico, finanzas, biblioteca, docencia, estudiante y reportes).

- **modelo:** entidades del negocio (`Usuario` y sus roles, `Pago`, `Libro`, `Prestamo`, `Inscripcion`, etc.) y clases de apoyo para consultas y reportes (`FilaMora`, `FilaPago`, `FilaIngreso`, entre otras).
- **persistencia:** subdividido en `interfaces` (contratos DAO) e `implementacion` (clases con el acceso JDBC).
- **transferencia:** objetos `Solicitud...` y `Respuesta...` para el intercambio de datos con el frontend, y clases de validación (`ValidacionUsuario`, `ValidacionEmpleado`, `ValidacionEstudiante`).
- **seguridad:** `FiltroAutenticacion` (valida el token y el rol), las anotaciones `@RolesPermitidos` y `@Publico`, y `AccesoMaestro`
- **excepciones:** las 34 excepciones personalizadas.

- **enums:** enumeraciones de estados y conceptos.
- **correo:** interfaz `EnviadorCorreo` e implementación por consola.
- **utilidades:** conexión a la base (`ConexionBase`), manejo de tokens (`JwtUtil`), contraseñas (`ContrasenaUtil`, `GeneradorContrasena`), códigos de recuperación (`GeneradorCodigo`), cálculo del dictamen (`Dictamen`) y generación de reportes PDF (`GeneradorReporte`).
- **resources:** recurso de comprobación del servicio (`PingResource`).
- La clase `JakartaRestConfiguration` configura la aplicación REST y registra las rutas.

Arquitectura del Frontend (Angular)

El frontend es una aplicación de página única (SPA) desarrollada con Angular 22 y TypeScript, que se ejecuta en `http://localhost:4200` y consume la API REST del backend. Está construida con componentes independientes (*standalone*), cada uno con su archivo de lógica (`.ts`), plantilla (`.html`), estilos (`.css`) y pruebas (`.spec.ts`).

Estructura de `src/app`

- **`app.routes.ts`:** define todas las rutas de la aplicación y los guards que las protegen.
- **`app.config.ts`:** configuración global de la aplicación, donde se registran el enrutador y el interceptor de sesión.
- **`configuracion.ts`:** contiene la constante `URL_API` con la dirección base del backend, de modo que cambiarla en un solo lugar basta para apuntar a otro servidor.
- **`menu.ts`:** define las opciones del menú lateral y las tarjetas de inicio por rol (`gruposPorRol`) y el color de acento de cada rol (`temaPorRol`).
- **`guards/`:** **`sesion-guard`** (exige haber iniciado sesión) y `rol-guard` (exige un rol permitido).
- **`interceptores/`:** **`interceptor-sesion`,** que adjunta el token a las peticiones.
- **`layout/marco/`:** el marco común de la aplicación, con menú lateral, barra superior y pie.
- **`servicios/`:** un servicio por módulo del backend (`pagos`, `inscripciones`, `libros`, `prestamos`, `reportes`, entre otros), que concentran las peticiones HTTP.
- **`paginas/`:** una carpeta por pantalla, nombradas por su función: `lista-*` (listados con búsqueda y acciones), `formulario-*` (creación y edición), **`reporte-*`**

(reportes), y las pantallas propias de cada rol (como `notas-maestro`, `perfil-estudiante` o `estado-cuenta`).

Enrutamiento y control de acceso

Las rutas se dividen en dos grupos. Las públicas son el inicio de sesión (`/login`), la bienvenida (`/`), la recuperación de contraseña (`/recuperar`) y la pantalla de acceso denegado (`/sin-acceso`). Las protegidas son hijas del componente `Marco`, que lleva el guard de sesión una sola vez y lo hereda a todas sus rutas.

Cada ruta protegida añade el `rolGuard` y declara en `data.roles` los roles que pueden entrar. Por ejemplo, `/grados` admite solo `SUPERADMIN`, `/empleados` admite `ADMIN` y `SUPERADMIN` (el SuperAdmin hereda las pantallas del Admin) y `/inscripciones` admite solo `SECRETARIA`. Un usuario con otro rol es enviado a `/sin-acceso`. Cualquier dirección que no exista redirige a `/login`.

Esta validación del frontend mejora la experiencia de uso, pero la seguridad real está en el backend, que vuelve a verificar el token y el rol en cada petición.

Sesión de usuario (servicio `Autenticacion`)

Este servicio concentra el manejo de la sesión: inicia sesión (`/auth/login`), solicita y restablece códigos de recuperación (`/auth/recuperacion/...`) y cambia la contraseña (`/auth/contrasena`). La respuesta de inicio de sesión contiene el token JWT, el identificador y nombre del usuario, su rol y si la contraseña es provisional.

La sesión se guarda en `sessionStorage` y no en `localStorage`, de modo que cada pestaña del navegador tiene su propia sesión y no se mezclan cuentas cuando dos usuarios abren el sistema en el mismo navegador. Cerrar sesión consiste en borrar esos datos del navegador.

Cuando la contraseña es provisional (por ejemplo, la de un estudiante recién inscrito), el sistema muestra una ventana (`modal-contrasena-provisional`) que solicita cambiarla; al hacerlo, el servicio apaga esa bandera en la sesión guardada.

Interceptor de sesión

Se ejecuta con cada petición HTTP que sale de Angular. Si la petición va dirigida al backend (`URL_API`), la clona y le agrega el encabezado `Authorization: Bearer <token>`; las peticiones hacia otras direcciones no reciben el token. Además, cuando el servidor responde `401` y existe una sesión guardada (el token venció o no es válido), el interceptor borra la sesión y redirige al inicio de sesión. El error se vuelve a lanzar para que también lo reciba la pantalla que hizo la petición.

Servicios y pantallas

Cada servicio expone métodos que devuelven *Observables*, que no ejecutan la petición hasta que la pantalla se suscribe. Las pantallas de tipo `lista-*` consultan al servicio, muestran los resultados y ofrecen las acciones de cada rol; las de tipo `formulario-*` validan los datos antes de enviarlos; las de tipo `reporte-*` permiten aplicar los filtros del reporte, verlo en pantalla y exportarlo a PDF.

Interfaz por rol

El marco construye el menú lateral con `gruposPorRol`, de modo que cada rol solo ve sus opciones. `temaPorRol` asigna a cada rol un color distintivo (SuperAdmin gris azulado, Admin verde, Secretaria azul, Bibliotecario ámbar, Maestro violeta y Estudiante turquesa) que el marco aplica como variables CSS. Los estilos se construyen con Tailwind CSS 4 y los iconos con Bootstrap Icons.

Patrones de Diseño

Arquitectura cliente-servidor en capas

El sistema se divide en un frontend (Angular) y un backend (API REST) que se comunican por HTTP con mensajes JSON. En el backend, las responsabilidades se separan en capas: los controladores REST reciben las peticiones, validan los datos y coordinan la lógica de negocio; las clases de persistencia acceden a la base de datos; y las clases de utilidades concentran funciones compartidas, como la generación de reportes y el manejo de tokens. Esta separación permite modificar la interfaz sin afectar la lógica de negocio, y optimizar el acceso a datos sin afectar los controladores.

Data Access Object (DAO)

Cada entidad del sistema cuenta con una clase de persistencia que encapsula toda la comunicación con la base de datos (por ejemplo, `ReporteDAOPersistencia`). Los controladores nunca ejecutan SQL directamente: delegan en estas clases, lo que mantiene separada la lógica de acceso a datos de la lógica de negocio.

Control de acceso declarativo mediante anotaciones

Los permisos se definen con la anotación personalizada `@RolesPermitidos`, colocada sobre las clases o los métodos de los controladores. Un filtro de Jersey intercepta cada petición, valida el token JWT y comprueba que el rol del usuario esté autorizado. Si la anotación se coloca en un método, esta reemplaza a la de la clase, lo que permite afinar permisos por operación.

Control de acceso por propiedad del recurso

Además del rol, ciertas operaciones verifican que el recurso pertenezca al usuario. Por ejemplo, un Maestro solo puede operar sobre los cursos y secciones que tiene asignados (`AccesoMaestro.asignacionPropia`), y un Estudiante solo consulta su propia información.

Manejo de excepciones personalizadas

El sistema define excepciones propias (por ejemplo, `ExcepcionPersistencia` y `ExcepcionReferenciaInvalida`) que representan errores de negocio o de datos. Los controladores las traducen a respuestas HTTP con mensajes claros para el usuario, sin exponer errores técnicos.

Transacciones

Las operaciones que modifican varias tablas al mismo tiempo (como inscribir a un estudiante, registrar un préstamo o una devolución con su multa, o anular un pago junto con su comprobante) se ejecutan dentro de una transacción con `commit` si todo sale bien y `rollback` si ocurre un error. En las operaciones sensibles a concurrencia, como validar el cupo de una sección, se utiliza bloqueo de filas con `SELECT ... FOR UPDATE`.

Generación de reportes basada en plantillas

Los reportes se construyen con plantillas `.jrxml` que se llenan con datos. La clase `GeneradorReporte` recibe la plantilla, los parámetros y las filas (como una colección de mapas) y devuelve el PDF, de modo que cada reporte solo debe preparar sus datos.

Aplicación de página única (SPA) con rutas protegidas

En Angular, las rutas se protegen con guards según el rol (`rolGuard`), el menú lateral se arma dinámicamente por rol y cada rol tiene su color distintivo.

Características Técnicas

- **Validación de datos en el servidor:** toda entrada se valida en el backend (formato, obligatoriedad y reglas de negocio), sin depender de las validaciones del frontend.
- **Prevención de inyección SQL:** todas las consultas usan sentencias preparadas (`PreparedStatement`).
- **Autenticación con JWT y contraseñas con BCrypt:** las contraseñas nunca se almacenan en texto plano. Los estudiantes inscritos reciben una contraseña provisional.
- **Recuperación de contraseña:** mediante un código temporal que expira y es de un solo uso.
- **Control de acceso por rol:** cada rol (SuperAdmin, Admin, Secretaria, Bibliotecario, Maestro y Estudiante) accede solo a las funciones que le corresponden, con herencia del SuperAdmin sobre los permisos del Admin.
- **Eliminación lógica:** los registros no se borran; se desactivan (estados Activo/Inactivo, Anulada, Anulado), conservando el historial.
- **Integridad en la base de datos:** claves foráneas, restricciones `CHECK` y columnas generadas que garantizan unicidad (por ejemplo, un solo pago vigente por concepto y un solo estudiante inscrito por año). Las restricciones de unicidad ignoran los registros anulados, por lo que un pago anulado puede cobrarse de nuevo.
- **Los montos se copian al registrarse:** el pago guarda el monto cobrado, por lo que cambiar una tarifa no altera los pagos anteriores.
- **Subida de archivos:** las portadas de los libros se cargan con formularios `multipart/form-data` y se almacenan en el servidor.
- **Reportes en PDF:** cinco reportes con plantillas propias, con logo y exportación con los mismos filtros aplicados.
- **Interfaz adaptada por rol:** color distintivo, menú dinámico y pantalla de inicio con accesos por módulo.

Descripción de Clases por Paquete

Por la cantidad de clases del sistema, se describen las de mayor relevancia de cada paquete. Las clases restantes siguen el mismo patrón: un controlador, una interfaz DAO y su implementación por cada entidad.

Paquete controlador

- **AutenticacionControlador:** gestiona el inicio de sesión, la solicitud de código de recuperación, el restablecimiento de contraseña y el cambio de contraseña. Es la puerta de entrada al sistema y emite el token JWT.
- **InscripcionControlador:** registra las inscripciones de estudiantes en una sección, generando su acceso cuando corresponde, y permite consultarlas y anularlas.
- **PagoControlador:** registra los pagos de inscripción, mensualidad, actividad y multa; los consulta y los anula; y emite sus comprobantes.
- **SolvenciaControlador:** verifica si un estudiante está solvente y emite la solvencia, con el límite de una por estudiante y por mes.
- **PrestamoControlador:** registra préstamos y devoluciones de libros, y genera la multa cuando un estudiante devuelve tarde.
- **LibroControlador:** administra el catálogo de libros, incluida la subida de portadas.
- **MaestroControlador:** concentra las operaciones del Maestro sobre sus cursos asignados: asistencia, actividades de zona, notas y examen final.
- **PanelEstudianteControlador:** atiende las consultas del Estudiante sobre su propia información: perfil, notas con dictamen, estado de cuenta, préstamos y multas.
- **ReporteControlador:** expone los cinco reportes (préstamos en mora, ingresos, historial de pagos, solventes y morosos, y rendimiento), cada uno en versión de consulta y de exportación a PDF.
- **AdminControlador, SuperAdminControlador, EmpleadoControlador, EstudianteControlador** y los de estructura académica (año lectivo, grado, carrera, curso, currículo, sección y asignación de maestros) y de finanzas (tarifa y actividad) ofrecen las operaciones de creación, edición, activación, desactivación y consulta de cada módulo.

Paquete modelo

- **Usuario:** clase base con los datos comunes de cualquier persona del sistema. De ella se especializan los roles.
- **Estudiante, Maestro, Admin, SuperAdmin, Secretaria y Bibliotecario:** representan a cada tipo de usuario con sus datos propios.
- **Empleado:** datos laborales del personal (puesto, fecha de contratación, salario).
- **AnioLectivo, Grado, Carrera, Curso, Curriculo, Seccion y AsignacionMaestro:** estructura académica del colegio.
- **Inscripcion:** vincula a un estudiante con una sección en un año lectivo.
- **Tarifa, Pago, Comprobante, Solvencia y ActividadExtracurricular:** módulo financiero.
- **Libro, Prestamo, Multa y ParametrosBiblioteca:** módulo de biblioteca.
- **ActividadZona, NotaActividad, NotaCurso y NotaTotal:** actividades y notas del estudiante.
- **Clases `Fila...`** (**FilaMora, FilaPago, FilaIngreso, FilaSolvencia, FilaCurso, FilaMejor**): representan una fila de los reportes, ya preparada para llenar las plantillas.

Paquete persistencia

- **DAOGenerico:** interfaz base que define las operaciones comunes de acceso a datos.
- **InscripcionDAOPersistencia:** inscribe al estudiante dentro de una transacción, validando el cupo de la sección con bloqueo de filas (`FOR UPDATE`) y creando el acceso si es necesario.
- **PagoDAOPersistencia:** registra pagos copiando el monto vigente de la tarifa, evita duplicados mediante las restricciones de la base de datos y anula pagos junto con su comprobante.
- **PrestamoDAOPersistencia:** controla préstamos y devoluciones, descuenta y repone ejemplares y genera la multa por atraso dentro de la misma transacción.
- **NotaDAOPersistencia y ZonaDAOPersistencia:** administran las actividades de zona y las notas, validando que la suma de la zona no pase de 70 puntos.
- **ReporteDAOPersistencia:** consultas de los cinco reportes.
- **PanelEstudianteDAOPersistencia:** consultas del Estudiante sobre su propia información.
- **UsuarioDAOPersistencia y CodigoRecuperacionDAOPersistencia:** autenticación, usuarios y códigos de recuperación.

Paquete seguridad

- **FiltroAutenticacion:** filtro que se ejecuta en cada petición; valida el token JWT y comprueba que el rol del usuario esté permitido en la ruta solicitada.
- **RolesPermitidos:** anotación que declara qué roles pueden acceder a una clase o método.
- **Publico:** anotación que marca los recursos accesibles sin autenticación (inicio de sesión y recuperación de contraseña).
- **AccesoMaestro:** verifica que un Maestro solo opere sobre los cursos y secciones que tiene asignados.

Paquete utilidades

- **ConexionBase:** obtiene las conexiones a MySQL desde el pool configurado en Tomcat.
- **JwtUtil:** genera y valida los tokens JWT.
- **ContrasenaUtil:** aplica el hash BCrypt y verifica contraseñas.
- **GeneradorContrasena y GeneradorCodigo:** crean las contraseñas provisionales y los códigos de recuperación.
- **Dictamen:** aplica la regla de aprobación (zona más examen final, con nota mínima de 60).
- **GeneradorReporte:** recibe la plantilla `.jrxml`, los parámetros y las filas, y devuelve el PDF del reporte.

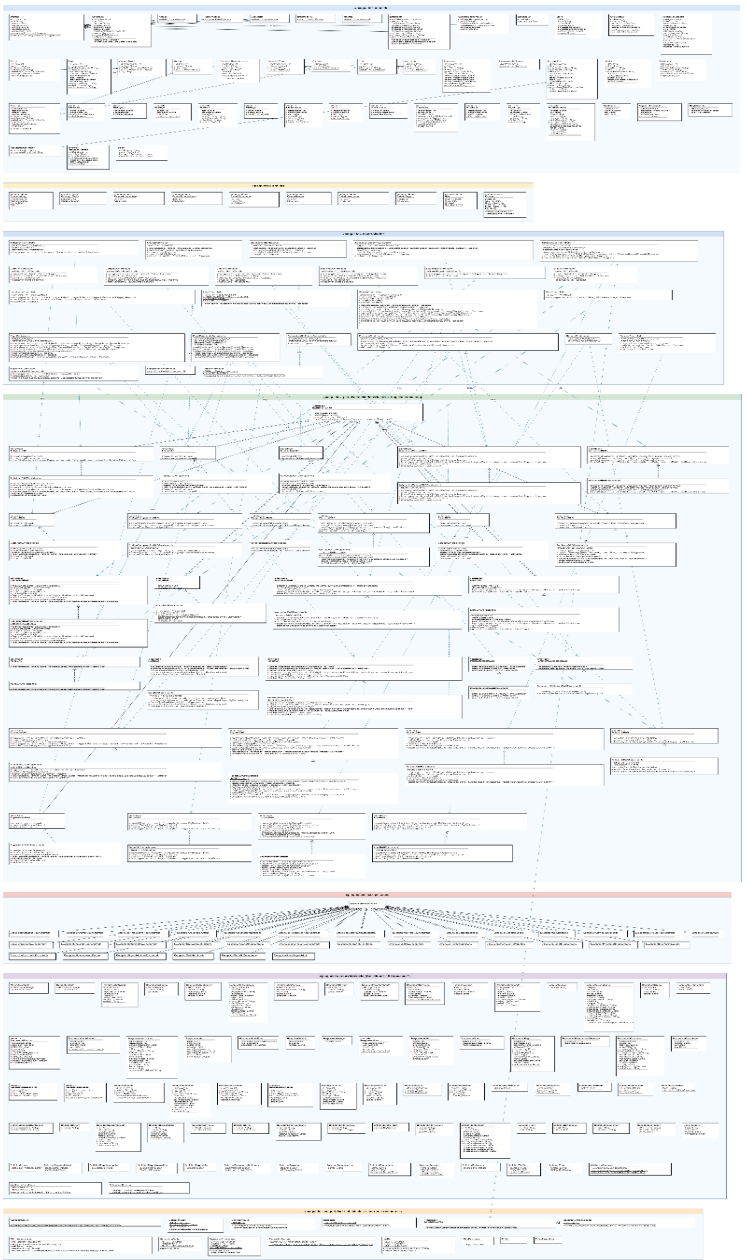
Paquete excepciones

Contiene 34 excepciones de negocio. Destacan `ExcepcionPersistencia` (error en el acceso a datos), `ExcepcionReferenciaInvalida` (datos que apuntan a registros inexistentes), `ExcepcionOperacionNoPermitida` (acción que viola una regla del sistema), `ExcepcionUltimoSuperAdmin` (impide quedarse sin SuperAdmin activo) y `ExcepcionAsignacionAjena` (un maestro intenta operar sobre un curso que no es suyo).

Diagramas del Sistema

Diagrama de Clases

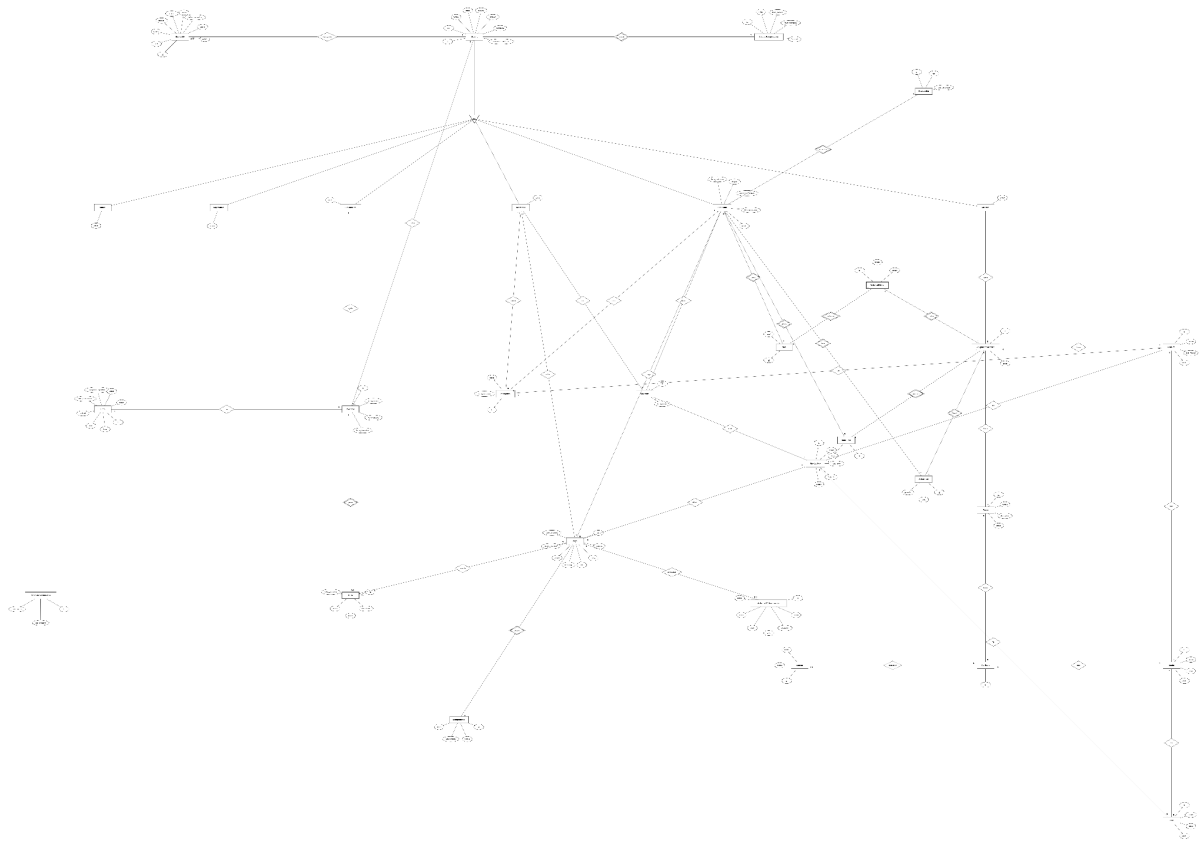
El diagrama de clases representa la estructura orientada a objetos del backend, organizada por paquetes: modelo (entidades del negocio), persistencia (interfaces DAO y sus implementaciones), controladores REST y seguridad. Se destaca la jerarquía de herencia de `Usuario` hacia sus roles y la relación entre cada controlador, su interfaz DAO y las entidades que maneja.



Para visualizar el diagrama completo también se puede acceder al siguiente enlace: https://drive.google.com/file/d/1EOw56vv7G3UiKkSctmFU1DfQ0nT_RcOw/view?usp=sharing

Diagrama Entidad-Relación

El diagrama entidad-relación representa el modelo de datos del sistema: las entidades, sus atributos y las relaciones entre ellas con su cardinalidad. Se muestra la especialización de `Usuario` en sus seis roles.



Para visualizar el diagrama completo también se puede acceder al siguiente enlace: https://drive.google.com/file/d/1L_RGYCM8QOAnx3ZLF4rKbmY_5GJ_V83V/view?usp=sharing

Diagrama de Tablas

El diagrama de tablas representa la estructura física final de la base de datos `sistema_colegio`, generado por ingeniería inversa en MySQL Workbench a partir del script SQL implementado. Incluye las claves primarias, las claves foráneas y las relaciones entre las 32 tablas que conforman el sistema.

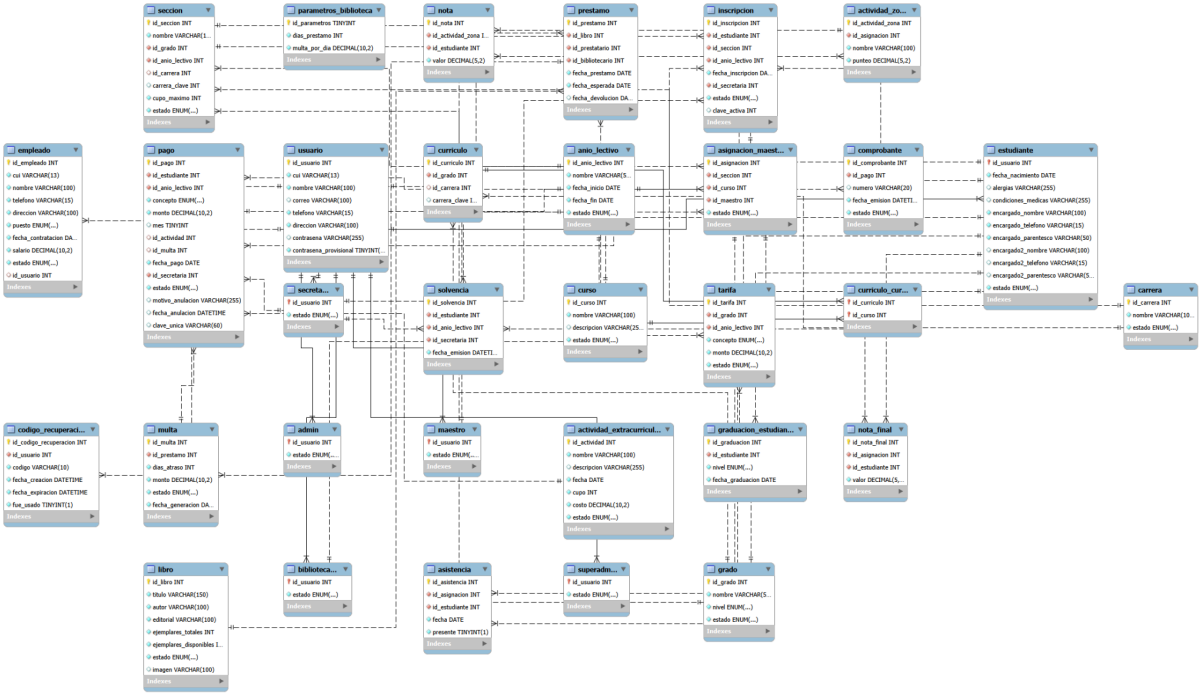


Diagrama de componentes

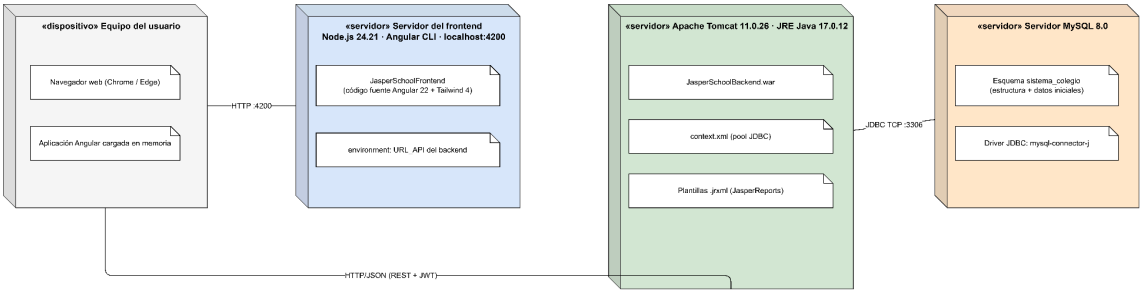
Muestra los componentes de las dos aplicaciones y cómo se comunican. El frontend en Angular consume el API REST del backend mediante HTTP/JSON con un token JWT. El backend accede a la base de datos MySQL por JDBC mediante un pool de conexiones.



Para visualizar el diagrama completo también se puede acceder al siguiente enlace: https://drive.google.com/file/d/1f_poA3-zZ_fQpFTYWB8kt2mRrZabAL7M/view?usp=sharing

Diagrama de despliegue

Muestra dónde se ejecuta cada parte del sistema: el navegador en el equipo del usuario, el frontend con Node.js en `localhost:4200`, el backend en Tomcat 11 con Java 17 y la base de datos en MySQL 8.



Para visualizar el diagrama completo también se puede acceder al siguiente enlace: https://drive.google.com/file/d/1wwcsJUsc7XIGUAVke0sRapSziHrdMeo5/view?usp=sharing

Mapeo Físico de la Base de Datos

La estrategia utilizada para representar la herencia de `Usuario` es la de "tabla por subclase" (*table per subclass*): la tabla `usuario` contiene los atributos comunes a todas las personas (CUI, nombre, correo, teléfono, dirección y contraseña), y cada rol cuenta con su propia tabla (`superadmin`, `admin`, `secretaria`, `bibliotecario`, `maestro` y `estudiante`) con sus datos específicos. Cada una se relaciona con la tabla padre mediante una clave foránea que a su vez es su clave primaria (`id_usuario`). El rol de una persona se determina por la tabla en la que aparece su registro.

Esta estrategia se eligió frente a una única tabla con todos los atributos posibles, o a una tabla independiente por cada rol, porque mantiene la base normalizada: se evitan columnas con valores nulos para atributos que no aplican a un rol y se conserva la relación de herencia mediante claves compartidas.

Otras decisiones de diseño de la base de datos:

- Eliminación lógica: los registros se desactivan o anulan (columnas `estado`) y no se borran, para conservar el historial.
- Unicidad con registros anulados: las columnas generadas `clave_unica` (en `pago`) y `clave_activa` (en `inscripcion`) valen `NULL` cuando el registro está anulado. Así un pago o una inscripción anulados no bloquean volver a registrar el mismo concepto.
- Montos históricos: el pago guarda el monto cobrado, copiado de la tarifa, actividad o multa. Cambiar una tarifa no altera los pagos anteriores.
- Restricciones `CHECK`: garantizan reglas como montos mayores a cero, notas dentro de rango (el examen final no supera 30 y las actividades de zona no pasan de 70) y la coherencia entre el concepto de un pago y sus columnas (mes, actividad o multa).
- Columnas generadas `carrera_clave`: en `curriculo` y `seccion` convierten el `NULL` de la carrera en `0` solo para la restricción de unicidad, ya que MySQL permite duplicados con `NULL`.

A continuación se presenta el script SQL completo de la estructura de la base de datos:

```
CREATE DATABASE IF NOT EXISTS sistema_colegio;

USE sistema_colegio;


-- ======================== BASE ========================


-- correo y contrasena son NULL: el estudiante se registra sin acceso, y la Secretaria se lo crea al inscribirlo

CREATE TABLE IF NOT EXISTS usuario (

    id_usuario INT AUTO_INCREMENT PRIMARY KEY,

    cui VARCHAR(13) NOT NULL UNIQUE,

    nombre VARCHAR(100) NOT NULL,

    correo VARCHAR(100) NULL UNIQUE,

    telefono VARCHAR(15) NOT NULL,

    direccion VARCHAR(100) NOT NULL,

    contrasena VARCHAR(255) NULL,

    contrasena_provisional BOOLEAN NOT NULL DEFAULT FALSE

);


CREATE TABLE IF NOT EXISTS admin (

    id_usuario INT PRIMARY KEY,

    estado ENUM('ACTIVO', 'INACTIVO') NOT NULL DEFAULT 'ACTIVO',

    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)

);


CREATE TABLE IF NOT EXISTS superadmin (
```

```
    id_usuario INT PRIMARY KEY,
    estado ENUM('ACTIVO', 'INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);

CREATE TABLE IF NOT EXISTS maestro (
    id_usuario INT NOT NULL,
    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    PRIMARY KEY (id_usuario),
    CONSTRAINT fk_maestro_usuario FOREIGN KEY (id_usuario)
REFERENCES usuario (id_usuario)
);

CREATE TABLE IF NOT EXISTS secretaria (
    id_usuario INT NOT NULL,
    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    PRIMARY KEY (id_usuario),
    CONSTRAINT fk_secretaria_usuario FOREIGN KEY (id_usuario)
REFERENCES usuario (id_usuario)
);

CREATE TABLE IF NOT EXISTS bibliotecario (
    id_usuario INT NOT NULL,
    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    PRIMARY KEY (id_usuario),
```

```
    CONSTRAINT fk_bibliotecario_usuario FOREIGN KEY (id_usuario)
REFERENCES usuario (id_usuario)

);


CREATE TABLE IF NOT EXISTS estudiante (

    id_usuario INT NOT NULL,

    fecha_nacimiento DATE NOT NULL,

    alergias VARCHAR(255) NULL,

    condiciones_medicas VARCHAR(255) NULL,

    encargado_nombre VARCHAR(100) NOT NULL,

    encargado_telefono VARCHAR(15) NOT NULL,

    encargado_parentesco VARCHAR(50) NOT NULL,

    encargado2_nombre VARCHAR(100) NULL,

    encargado2_telefono VARCHAR(15) NULL,

    encargado2_parentesco VARCHAR(50) NULL,

    estado ENUM('ACTIVO','INACTIVO','GRADUADO') NOT NULL DEFAULT
'ACTIVO',

    PRIMARY KEY (id_usuario),

    CONSTRAINT fk_estudiante_usuario FOREIGN KEY (id_usuario)
REFERENCES usuario (id_usuario),

    -- el segundo encargado se llena completo o no se llena

    CONSTRAINT ck_encargado2_completo CHECK (

        (encargado2_nombre IS NULL AND encargado2_telefono IS NULL AND
encargado2_parentesco IS NULL)

        OR (encargado2_nombre IS NOT NULL AND encargado2_telefono IS
NOT NULL AND encargado2_parentesco IS NOT NULL))

);
```

```
CREATE TABLE IF NOT EXISTS graduacion_estudiante (

    id_graduacion INT NOT NULL AUTO_INCREMENT,

    id_estudiante INT NOT NULL,

    nivel ENUM('PRE_PRIMARIA','PRIMARIA','BASICO','DIVERSIFICADO')
NOT NULL,

    fecha_graduacion DATE NOT NULL,

    PRIMARY KEY (id_graduacion),

    UNIQUE KEY uq_graduacion (id_estudiante, nivel),

    CONSTRAINT fk_graduacion_estudiante FOREIGN KEY (id_estudiante)
REFERENCES estudiante (id_usuario)

);


CREATE TABLE IF NOT EXISTS empleado (

    id_empleado INT NOT NULL AUTO_INCREMENT,

    cui VARCHAR(13) NOT NULL,

    nombre VARCHAR(100) NOT NULL,

    telefono VARCHAR(15) NOT NULL,

    direccion VARCHAR(100) NOT NULL,

    puesto
ENUM('MAESTRO','SECRETARIA','BIBLIOTECARIO','ADMINISTRATIVO','MA
NTENIMIENTO','LIMPIEZA') NOT NULL,

    fecha_contratacion DATE NOT NULL,

    salario DECIMAL(10,2) NOT NULL,

    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',

    id_usuario INT NULL,

    PRIMARY KEY (id_empleado),
```

```
    UNIQUE KEY uq_empleado_cui (cui),
    UNIQUE KEY uq_empleado_usuario (id_usuario),
    CONSTRAINT fk_empleado_usuario FOREIGN KEY (id_usuario)
REFERENCES usuario (id_usuario),
    CONSTRAINT ck_empleado_salario CHECK (salario > 0)
);

CREATE TABLE IF NOT EXISTS codigo_recuperacion (
    id_codigo_recuperacion INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    codigo VARCHAR(10) NOT NULL,
    fecha_creacion DATETIME NOT NULL,
    fecha_expiracion DATETIME NOT NULL,
    fue_usado BOOLEAN NOT NULL DEFAULT FALSE,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);

-- ========================= ACADEMICO
=========================

CREATE TABLE IF NOT EXISTS anio_lectivo (
    id_anio_lectivo INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    estado ENUM('ACTIVO', 'CERRADO') NOT NULL DEFAULT 'ACTIVO'
```

```
);

CREATE TABLE IF NOT EXISTS grado (
    id_grado INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    nivel ENUM('PRE_PRIMARIA', 'PRIMARIA', 'BASICO', 'DIVERSIFICADO')
NOT NULL,
    estado ENUM('ACTIVO', 'INACTIVO') NOT NULL DEFAULT 'ACTIVO'
);

CREATE TABLE IF NOT EXISTS carrera (
    id_carrera INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    estado ENUM('ACTIVO', 'INACTIVO') NOT NULL DEFAULT 'ACTIVO'
);

CREATE TABLE IF NOT EXISTS curso (
    id_curso INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    descripcion VARCHAR(255) NULL,
    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    PRIMARY KEY (id_curso),
    UNIQUE KEY uq_curso_nombre (nombre)
);
```

```
-- carrera_clave convierte el NULL en 0 SOLO para la restriccion UNIQUE:

-- sin ella, MySQL permitiria dos curriculos del mismo grado sin carrera

CREATE TABLE IF NOT EXISTS curriculo (

    id_curriculo INT NOT NULL AUTO_INCREMENT,

    id_grado INT NOT NULL,

    id_carrera INT NULL,

    carrera_clave INT GENERATED ALWAYS AS (IFNULL(id_carrera, 0))
STORED,

    PRIMARY KEY (id_curriculo),

    UNIQUE KEY uq_curriculo (id_grado, carrera_clave),

    CONSTRAINT fk_curriculo_grado FOREIGN KEY (id_grado)
REFERENCES grado (id_grado),

    CONSTRAINT fk_curriculo_carrera FOREIGN KEY (id_carrera)
REFERENCES carrera (id_carrera)

);


CREATE TABLE IF NOT EXISTS curriculo_curso (

    id_curriculo INT NOT NULL,

    id_curso INT NOT NULL,

    PRIMARY KEY (id_curriculo, id_curso),

    CONSTRAINT fk_cc_curriculo FOREIGN KEY (id_curriculo) REFERENCES
curriculo (id_curriculo),

    CONSTRAINT fk_cc_curso FOREIGN KEY (id_curso) REFERENCES curso
(id_curso)

);


CREATE TABLE IF NOT EXISTS seccion (
```

```
    id_seccion INT NOT NULL AUTO_INCREMENT,

    nombre VARCHAR(10) NOT NULL,

    id_grado INT NOT NULL,

    id_anio_lectivo INT NOT NULL,

    id_carrera INT NULL,

    carrera_clave INT GENERATED ALWAYS AS (IFNULL(id_carrera, 0))
STORED,

    cupo_maximo INT NOT NULL,

    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',

    PRIMARY KEY (id_seccion),

    UNIQUE KEY uq_seccion (id_grado, id_anio_lectivo, carrera_clave,
nombre),

    CONSTRAINT fk_seccion_grado FOREIGN KEY (id_grado) REFERENCES
grado (id_grado),

    CONSTRAINT fk_seccion_anio FOREIGN KEY (id_anio_lectivo)
REFERENCES anio_lectivo (id_anio_lectivo),

    CONSTRAINT fk_seccion_carrera FOREIGN KEY (id_carrera)
REFERENCES carrera (id_carrera),

    CONSTRAINT ck_seccion_cupo CHECK (cupo_maximo > 0)

);



-- un solo maestro por curso y seccion

CREATE TABLE IF NOT EXISTS asignacion_maestro (

    id_asignacion INT NOT NULL AUTO_INCREMENT,

    id_seccion INT NOT NULL,

    id_curso INT NOT NULL,

    id_maestro INT NOT NULL,
```

```
    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',

    PRIMARY KEY (id_asignacion),

    UNIQUE KEY uq_asignacion (id_seccion, id_curso),

    CONSTRAINT fk_asignacion_seccion FOREIGN KEY (id_seccion)
REFERENCES seccion (id_seccion),

    CONSTRAINT fk_asignacion_curso FOREIGN KEY (id_curso)
REFERENCES curso (id_curso),

    CONSTRAINT fk_asignacion_maestro FOREIGN KEY (id_maestro)
REFERENCES maestro (id_usuario)

);



-- ========================= FINANZAS
=========================



CREATE TABLE IF NOT EXISTS tarifa (

    id_tarifa INT NOT NULL AUTO_INCREMENT,

    id_grado INT NOT NULL,

    id_anio_lectivo INT NOT NULL,

    concepto ENUM('INSCRIPCION','MENSUALIDAD') NOT NULL,

    monto DECIMAL(10,2) NOT NULL,

    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',

    PRIMARY KEY (id_tarifa),

    UNIQUE KEY uq_tarifa (id_grado, id_anio_lectivo, concepto),

    CONSTRAINT fk_tarifa_grado FOREIGN KEY (id_grado) REFERENCES
grado (id_grado),

    CONSTRAINT fk_tarifa_anio FOREIGN KEY (id_anio_lectivo)
REFERENCES anio_lectivo (id_anio_lectivo),

    CONSTRAINT ck_tarifa_monto CHECK (monto > 0)
```

```
);


-- costo = 0 significa actividad gratuita
CREATE TABLE IF NOT EXISTS actividad_extracurricular (
    id_actividad INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    descripcion VARCHAR(255) NULL,
    fecha DATE NOT NULL,
    cupo INT NOT NULL,
    costo DECIMAL(10,2) NOT NULL DEFAULT 0,
    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    PRIMARY KEY (id_actividad),
    UNIQUE KEY uq_actividad (nombre, fecha),
    CONSTRAINT ck_actividad_cupo CHECK (cupo > 0),
    CONSTRAINT ck_actividad_costo CHECK (costo >= 0)
);


-- clave_activa es NULL en inscripciones anuladas: quien se anula puede
reinscribirse en el mismo año
CREATE TABLE IF NOT EXISTS inscripcion (
    id_inscripcion INT NOT NULL AUTO_INCREMENT,
    id_estudiante INT NOT NULL,
    id_seccion INT NOT NULL,
    id_anio_lectivo INT NOT NULL,
    fecha_inscripcion DATE NOT NULL,
```

```
    id_secretaria INT NOT NULL,

    estado ENUM('ACTIVA','ANULADA') NOT NULL DEFAULT 'ACTIVA',

    clave_activa INT GENERATED ALWAYS AS (IF(estado = 'ACTIVA',
id_estudiante, NULL)) STORED,

    PRIMARY KEY (id_inscripcion),

    UNIQUE KEY uq_inscripcion_activa (id_anio_lectivo, clave_activa),

    CONSTRAINT fk_inscripcion_estudiante FOREIGN KEY (id_estudiante)
REFERENCES estudiante (id_usuario),

    CONSTRAINT fk_inscripcion_seccion FOREIGN KEY (id_seccion)
REFERENCES seccion (id_seccion),

    CONSTRAINT fk_inscripcion_anio FOREIGN KEY (id_anio_lectivo)
REFERENCES anio_lectivo (id_anio_lectivo),

    CONSTRAINT fk_inscripcion_secretaria FOREIGN KEY (id_secretaria)
REFERENCES secretaria (id_usuario)

);



-- ========================= BIBLIOTECA
=========================



-- una sola fila; los valores iniciales son de ejemplo

CREATE TABLE IF NOT EXISTS parametros_biblioteca (

    id_parametros TINYINT NOT NULL,

    dias_prestamo INT NOT NULL,

    multa_por_dia DECIMAL(10,2) NOT NULL,

    PRIMARY KEY (id_parametros),

    CONSTRAINT ck_parametros_una_fila CHECK (id_parametros = 1),

    CONSTRAINT ck_parametros_dias CHECK (dias_prestamo > 0),

    CONSTRAINT ck_parametros_multa CHECK (multa_por_dia >= 0)
```

```
);


INSERT IGNORE INTO parametros_biblioteca (id_parametros, dias_prestamo,
multa_por_dia)

VALUES (1, 7, 5.00);


-- imagen: nombre del archivo de la portada, guardada en
C:\jasper-imagenes\libros

CREATE TABLE IF NOT EXISTS libro (

    id_libro INT NOT NULL AUTO_INCREMENT,

    titulo VARCHAR(150) NOT NULL,

    autor VARCHAR(100) NOT NULL,

    editorial VARCHAR(100) NOT NULL,

    ejemplares_totales INT NOT NULL,

    ejemplares_disponibles INT NOT NULL,

    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',

    imagen VARCHAR(100) NULL,

    PRIMARY KEY (id_libro),

    UNIQUE KEY uq_libro (titulo, autor, editorial),

    CONSTRAINT ck_libro_totales CHECK (ejemplares_totales > 0),

    CONSTRAINT ck_libro_disponibles CHECK (ejemplares_disponibles >= 0
AND ejemplares_disponibles <= ejemplares_totales)

);


-- el prestatario es un usuario: estudiante o maestro (se distingue por la tabla
donde esta)

CREATE TABLE IF NOT EXISTS prestamo (
```

```
    id_prestamo INT NOT NULL AUTO_INCREMENT,

    id_libro INT NOT NULL,

    id_prestatario INT NOT NULL,

    id_bibliotecario INT NOT NULL,

    fecha_prestamo DATE NOT NULL,

    fecha_esperada DATE NOT NULL,

    fecha_devolucion DATE NULL,

    PRIMARY KEY (id_prestamo),

    KEY ix_prestamo_prestatario (id_prestatario, fecha_devolucion),

    CONSTRAINT fk_prestamo_libro FOREIGN KEY (id_libro) REFERENCES
libro (id_libro),

    CONSTRAINT fk_prestamo_prestatario FOREIGN KEY (id_prestatario)
REFERENCES usuario (id_usuario),

    CONSTRAINT fk_prestamo_bibliotecario FOREIGN KEY (id_bibliotecario)
REFERENCES bibliotecario (id_usuario),

    CONSTRAINT ck_prestamo_fechas CHECK (fecha_esperada >=
fecha_prestamo)

);



-- una multa por prestamo, solo cuando un estudiante devuelve tarde. El monto
se copia al generarla

CREATE TABLE IF NOT EXISTS multa (

    id_multa INT NOT NULL AUTO_INCREMENT,

    id_prestamo INT NOT NULL,

    dias_atraso INT NOT NULL,

    monto DECIMAL(10,2) NOT NULL,

    estado ENUM('PENDIENTE','PAGADA') NOT NULL DEFAULT
'PENDIENTE',
```

```
    fecha_generacion DATE NOT NULL,

    PRIMARY KEY (id_multa),

    UNIQUE KEY uq_multa_prestamo (id_prestamo),

    CONSTRAINT fk_multa_prestamo FOREIGN KEY (id_prestamo)
REFERENCES prestamo (id_prestamo),

    CONSTRAINT ck_multa_dias CHECK (dias_atraso > 0),

    CONSTRAINT ck_multa_monto CHECK (monto > 0)

);



-- ======================== PAGOS ========================


-- monto = lo cobrado (copiado de la tarifa, actividad o multa): cambiar la tarifa
no altera pagos viejos.

-- clave_unica es NULL en pagos anulados: un pago anulado no bloquea
volver a cobrar el mismo concepto.

CREATE TABLE IF NOT EXISTS pago (

    id_pago INT NOT NULL AUTO_INCREMENT,

    id_estudiante INT NOT NULL,

    id_anio_lectivo INT NOT NULL,

    concepto ENUM('INSCRIPCION','MENSUALIDAD','ACTIVIDAD','MULTA')
NOT NULL,

    monto DECIMAL(10,2) NOT NULL,

    mes TINYINT NULL,

    id_actividad INT NULL,

    id_multa INT NULL,

    fecha_pago DATE NOT NULL,

    id_secretaria INT NOT NULL,
```

```
    estado ENUM('VIGENTE','ANULADO') NOT NULL DEFAULT 'VIGENTE',

    motivo_anulacion VARCHAR(255) NULL,

    fecha_anulacion DATETIME NULL,

    clave_unica VARCHAR(60) GENERATED ALWAYS AS (

        CASE WHEN estado <> 'VIGENTE' THEN NULL

            WHEN concepto = 'INSCRIPCION' THEN CONCAT('I-', id_estudiante,
'-', id_anio_lectivo)

            WHEN concepto = 'MENSUALIDAD' THEN CONCAT('M-',
id_estudiante, '-', id_anio_lectivo, '-', mes)

            WHEN concepto = 'ACTIVIDAD' THEN CONCAT('A-', id_estudiante, '-',
id_actividad)

            ELSE CONCAT('U-', id_multa) END) STORED,

    PRIMARY KEY (id_pago),

    UNIQUE KEY uq_pago_vigente (clave_unica),

    KEY ix_pago_estudiante (id_estudiante, fecha_pago),

    CONSTRAINT fk_pago_estudiante FOREIGN KEY (id_estudiante)
REFERENCES estudiante (id_usuario),

    CONSTRAINT fk_pago_anio FOREIGN KEY (id_anio_lectivo)
REFERENCES anio_lectivo (id_anio_lectivo),

    CONSTRAINT fk_pago_actividad FOREIGN KEY (id_actividad)
REFERENCES actividad_extracurricular (id_actividad),

    CONSTRAINT fk_pago_multa FOREIGN KEY (id_multa) REFERENCES
multa (id_multa),

    CONSTRAINT fk_pago_secretaria FOREIGN KEY (id_secretaria)
REFERENCES secretaria (id_usuario),

    CONSTRAINT ck_pago_monto CHECK (monto >= 0),

    CONSTRAINT ck_pago_mes CHECK ((concepto = 'MENSUALIDAD' AND
mes BETWEEN 1 AND 12)

                                  OR (concepto <> 'MENSUALIDAD' AND mes IS NULL)),
```

```
   CONSTRAINT ck_pago_actividad CHECK ((concepto = 'ACTIVIDAD' AND
id_actividad IS NOT NULL)

                                OR (concepto <> 'ACTIVIDAD' AND id_actividad IS
NULL)),

   CONSTRAINT ck_pago_multa CHECK ((concepto = 'MULTA' AND id_multa
IS NOT NULL)

                               OR (concepto <> 'MULTA' AND id_multa IS NULL)),

   CONSTRAINT ck_pago_anulacion CHECK ((estado = 'ANULADO' AND
motivo_anulacion IS NOT NULL)

                                OR (estado = 'VIGENTE' AND motivo_anulacion IS
NULL))

);


-- uno por pago; se anula junto con el pago

CREATE TABLE IF NOT EXISTS comprobante (

   id_comprobante INT NOT NULL AUTO_INCREMENT,

   id_pago INT NOT NULL,

   numero VARCHAR(20) NULL,

   fecha_emision DATETIME NOT NULL,

   estado ENUM('EMITIDO','ANULADO') NOT NULL DEFAULT 'EMITIDO',

   PRIMARY KEY (id_comprobante),

   UNIQUE KEY uq_comprobante_pago (id_pago),

   UNIQUE KEY uq_comprobante_numero (numero),

   CONSTRAINT fk_comprobante_pago FOREIGN KEY (id_pago)
REFERENCES pago (id_pago)

);


-- se guarda cada emision (una por estudiante y mes, lo controla el backend)
```

```
CREATE TABLE IF NOT EXISTS solvencia (

    id_solvencia INT NOT NULL AUTO_INCREMENT,

    id_estudiante INT NOT NULL,

    id_anio_lectivo INT NOT NULL,

    id_secretaria INT NOT NULL,

    fecha_emision DATETIME NOT NULL,

    PRIMARY KEY (id_solvencia),

    CONSTRAINT fk_solvencia_estudiante FOREIGN KEY (id_estudiante)
REFERENCES estudiante (id_usuario),

    CONSTRAINT fk_solvencia_anio FOREIGN KEY (id_anio_lectivo)
REFERENCES anio_lectivo (id_anio_lectivo),

    CONSTRAINT fk_solvencia_secretaria FOREIGN KEY (id_secretaria)
REFERENCES secretaria (id_usuario)

);



-- ========================= ITERACION 4: DOCENCIA Y NOTAS
=========================



-- una fila por asignacion, fecha y estudiante: presente o ausente

CREATE TABLE IF NOT EXISTS asistencia (

    id_asistencia INT NOT NULL AUTO_INCREMENT,

    id_asignacion INT NOT NULL,

    id_estudiante INT NOT NULL,

    fecha DATE NOT NULL,

    presente BOOLEAN NOT NULL,

    PRIMARY KEY (id_asistencia),

    UNIQUE KEY uq_asistencia (id_asignacion, fecha, id_estudiante),
```

```
    CONSTRAINT fk_asistencia_asignacion FOREIGN KEY (id_asignacion)
REFERENCES asignacion_maestro (id_asignacion),

    CONSTRAINT fk_asistencia_estudiante FOREIGN KEY (id_estudiante)
REFERENCES estudiante (id_usuario)

);


-- la suma de punteos de una asignacion no pasa de 70 (lo valida el backend);
el examen final vale 30

CREATE TABLE IF NOT EXISTS actividad_zona (

    id_actividad_zona INT NOT NULL AUTO_INCREMENT,

    id_asignacion INT NOT NULL,

    nombre VARCHAR(100) NOT NULL,

    punteo DECIMAL(5,2) NOT NULL,

    PRIMARY KEY (id_actividad_zona),

    UNIQUE KEY uq_actividad_zona (id_asignacion, nombre),

    CONSTRAINT fk_zona_asignacion FOREIGN KEY (id_asignacion)
REFERENCES asignacion_maestro (id_asignacion),

    CONSTRAINT ck_zona_punteo CHECK (punteo > 0 AND punteo <= 70)

);


-- nota de un estudiante en una actividad de zona (0 hasta el punteo, lo valida
el backend)

CREATE TABLE IF NOT EXISTS nota (

    id_nota INT NOT NULL AUTO_INCREMENT,

    id_actividad_zona INT NOT NULL,

    id_estudiante INT NOT NULL,

    valor DECIMAL(5,2) NOT NULL,
```

```
    PRIMARY KEY (id_nota),

    UNIQUE KEY uq_nota (id_actividad_zona, id_estudiante),

    CONSTRAINT fk_nota_actividad FOREIGN KEY (id_actividad_zona)
REFERENCES actividad_zona (id_actividad_zona),

    CONSTRAINT fk_nota_estudiante FOREIGN KEY (id_estudiante)
REFERENCES estudiante (id_usuario),

    CONSTRAINT ck_nota_valor CHECK (valor >= 0)

);



-- examen final: de 0 a 30, una nota por estudiante y asignacion

CREATE TABLE IF NOT EXISTS nota_final (

    id_nota_final INT NOT NULL AUTO_INCREMENT,

    id_asignacion INT NOT NULL,

    id_estudiante INT NOT NULL,

    valor DECIMAL(5,2) NOT NULL,

    PRIMARY KEY (id_nota_final),

    UNIQUE KEY uq_nota_final (id_asignacion, id_estudiante),

    CONSTRAINT fk_nf_asignacion FOREIGN KEY (id_asignacion)
REFERENCES asignacion_maestro (id_asignacion),

    CONSTRAINT fk_nf_estudiante FOREIGN KEY (id_estudiante)
REFERENCES estudiante (id_usuario),

    CONSTRAINT ck_nf_valor CHECK (valor >= 0 AND valor <= 30)

);
```

Pseudocódigo de los Procesos más Importantes

1. Inicio de sesión (AutenticacionControlador / UsuarioDAOPersistencia)

```
FUNCION iniciarSesion(correo, contrasena)
   usuario = buscar usuario por correo

   SI usuario no existe O su estado es inactivo ENTONCES
      lanzar ExcepcionCuentaInvalida
   FIN SI

   SI NO verificarBCrypt(contrasena, usuario.hash) ENTONCES
      lanzar ExcepcionCuentaInvalida
   FIN SI

   rol = determinar rol según la tabla donde aparece el usuario
   token = generarJWT(usuario.id, rol)

   retornar token, rol y si la contraseña es provisional
FIN FUNCION
```

2. Validación de acceso en cada petición (FiltroAutenticacion)

```
FUNCION filtrar(peticion)
   SI el recurso está marcado como @Publico ENTONCES
      permitir la petición
      retornar
   FIN SI

   token = leer encabezado Authorization
   SI el token no existe O no es válido O expiró ENTONCES
      responder 401 (no autenticado)
   FIN SI

   rolesPermitidos = leer @RolesPermitidos del método (o de la clase si el método
no la tiene)
   SI el rol del token NO está en rolesPermitidos ENTONCES
      responder 403 (sin permiso)
   FIN SI

   permitir la petición
FIN FUNCION
```

3. Inscripción de un estudiante (InscripcionDAOPersistencia)

```
FUNCION inscribir(idEstudiante, idSeccion, correo, contrasenaProvisional)
   validar que el estudiante esté activo
   validar que la sección esté activa y pertenezca al año lectivo activo

   INICIAR TRANSACCION
      bloquear la fila de la sección (SELECT ... FOR UPDATE)

      SI el estudiante ya tiene una inscripción activa en ese año ENTONCES
         deshacer; lanzar ExcepcionOperacionNoPermitida
      FIN SI

      inscritos = contar inscripciones activas de la sección
      SI inscritos >= cupoMaximo ENTONCES
         deshacer; lanzar ExcepcionOperacionNoPermitida ("sin cupo")
      FIN SI

      SI el estudiante aún no tiene acceso ENTONCES
         SI el correo ya existe ENTONCES
            deshacer; lanzar ExcepcionCorreoDuplicado
         FIN SI
         guardar correo y hash BCrypt de la contraseña provisional
         marcar la contraseña como provisional
      FIN SI

      registrar la inscripción con la fecha actual y la Secretaria que la realiza
   CONFIRMAR TRANSACCION
FIN FUNCION
```

4. Devolución de un libro con cálculo de multa (PrestamoDAOPersistencia)

```
FUNCION registrarDevolucion(idPrestamo)
   INICIAR TRANSACCION
      prestamo = buscar préstamo sin devolver (bloqueado con FOR UPDATE)
      SI no existe ENTONCES
         deshacer; lanzar ExcepcionPrestamoNoEncontrado
      FIN SI

      guardar la fecha de devolución = fecha actual
      sumar un ejemplar disponible al libro

      SI fechaActual > prestamo.fechaEsperada ENTONCES
```

```
        diasAtraso = fechaActual - prestamo.fechaEsperada

        SI el prestatario es un estudiante ENTONCES
          multaPorDia = leer de los parámetros de biblioteca
          monto = diasAtraso * multaPorDia
          registrar multa PENDIENTE con diasAtraso y monto
        SINO
          (maestro: se registra el atraso, no se genera multa)
        FIN SI
      FIN SI
    CONFIRMAR TRANSACCION
FIN FUNCION
```

5. Generación de solvencia (SolvenciaDAOPersistencia)

```
FUNCION generarSolvencia(idEstudiante)
    validar que el estudiante esté inscrito en el año lectivo activo

    SI ya existe una solvencia de ese estudiante en el mes actual ENTONCES
      retornar "ya existe una solvencia este mes"
    FIN SI

    adeudado = 0

    SI NO existe pago vigente de la inscripción ENTONCES
      adeudado = adeudado + tarifa de inscripción de su grado
    FIN SI

    PARA CADA mes desde el inicio del año lectivo hasta el mes actual
      estaVencido = (mes es anterior al mes actual) O (mes es el actual Y día > 10)
      SI estaVencido Y NO existe pago vigente de esa mensualidad ENTONCES
        adeudado = adeudado + tarifa de mensualidad de su grado
      FIN SI
    FIN PARA

    SI adeudado > 0 ENTONCES
      retornar "no solvente" con el monto adeudado
    SINO
      guardar la solvencia con la fecha de emisión
      retornar la solvencia
    FIN SI
FIN FUNCION
```

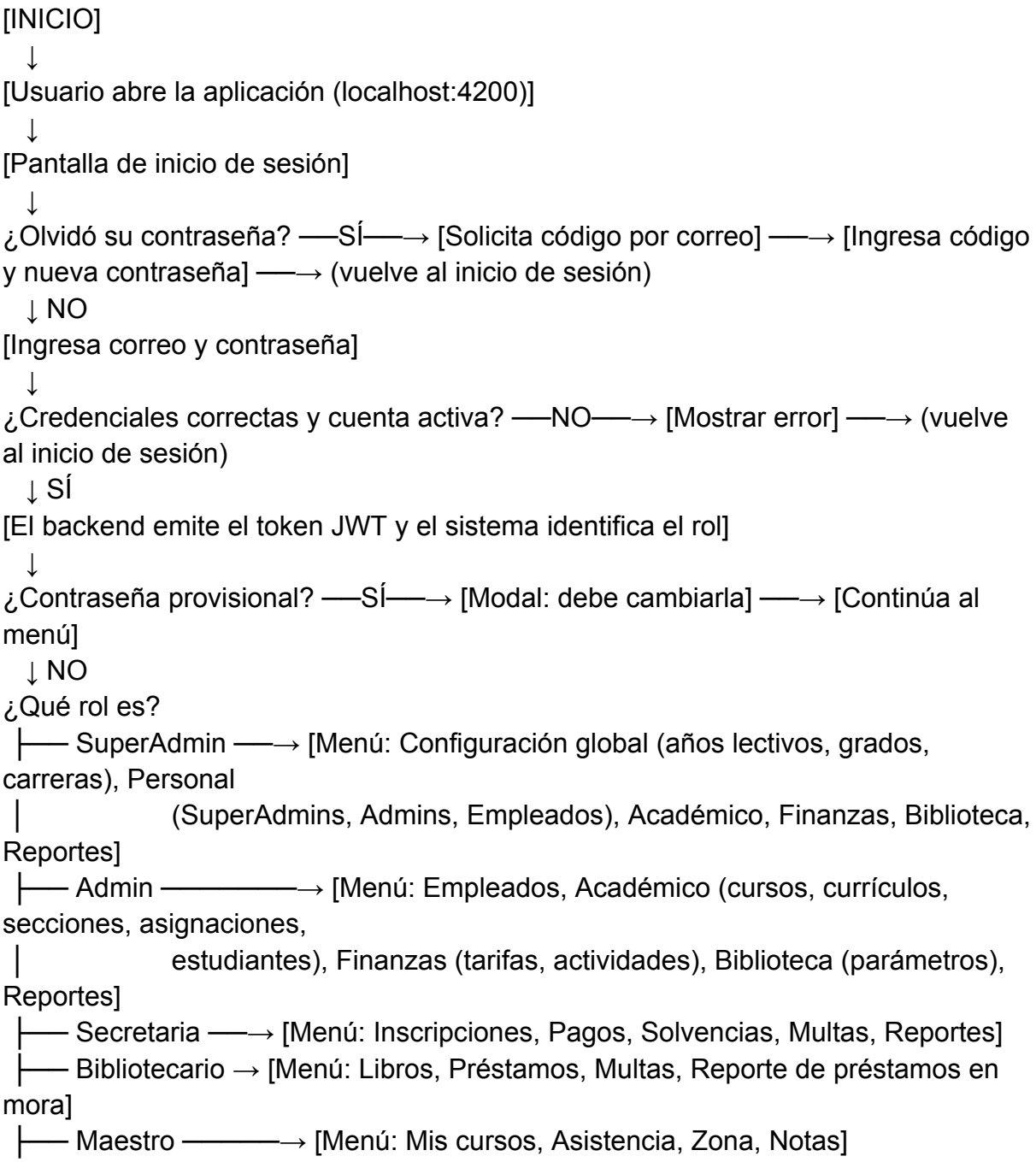
6. Cálculo del dictamen (Dictamen)

```
FUNCION calcularDictamen(totalZona, notaExamenFinal)
   total = totalZona + notaExamenFinal       // sobre 100

   SI total >= 60 ENTONCES
      retornar "APROBADO"
   SINO
      retornar "REPROBADO"
   FIN SI
FIN FUNCION
```

Diagrama de Flujo

El diagrama de flujo representa el recorrido general de un usuario dentro del sistema, desde el primer acceso hasta la realización de una acción según su rol. Tras autenticarse, el sistema identifica el rol y presenta el menú correspondiente. Cada acción pasa por un proceso de validación de datos y reglas de negocio antes de registrarse en la base de datos, y cada petición es verificada por el filtro de autenticación, que comprueba el token y el rol.



└── Estudiante ──→ [Menú: Mi perfil, Mis notas, Estado de cuenta, Préstamos y multas, Biblioteca]
↓
[Usuario realiza la acción correspondiente]
↓
[El backend verifica el token y el rol permitido]
↓
¿Autorizado? ──NO──→ [Respuesta de acceso denegado]
↓ SÍ
[El sistema valida los datos y las reglas de negocio]
↓
¿Validación exitosa? ──NO──→ [Mostrar mensaje de error] ──→ (vuelve al formulario)
↓ SÍ
[El sistema registra o consulta la información en la base de datos (con transacción si afecta varias tablas)]
↓
¿Es un reporte? ──SÍ──→ [Se muestra en pantalla y puede exportarse a PDF]
↓ NO
[El sistema muestra la confirmación al usuario]
↓
[FIN o vuelve al menú principal]

Conclusión

El desarrollo del Sistema de Gestión Escolar de Jasper International School permitió aplicar de manera integral los conocimientos adquiridos en el curso de Introducción a la Programación y Computación 2, desde el análisis de requerimientos mediante casos de uso hasta la implementación completa de una aplicación web con una arquitectura cliente-servidor desacoplada: un backend con API REST en Java y un frontend en Angular.

La organización del backend en capas (controladores, persistencia, modelo, seguridad y utilidades) permitió incorporar de manera progresiva los módulos del sistema (estructura académica, personal, inscripciones, pagos, biblioteca, docencia, autoservicio del estudiante y reportes) a lo largo de cuatro iteraciones, sin comprometer la estabilidad de lo ya implementado. El uso de interfaces para la capa de acceso a datos, objetos de transferencia para la comunicación con el frontend y excepciones propias facilitó el mantenimiento y la claridad del código.

Asimismo, la seguridad se resolvió con autenticación mediante tokens JWT, contraseñas protegidas con BCrypt y un control de acceso por roles que valida cada petición. Las transacciones y las restricciones de la base de datos garantizaron la consistencia de la información en las operaciones que afectan varias tablas, como las inscripciones, los pagos y las devoluciones de libros. La generación de reportes en PDF con JasperReports permitió ofrecer a cada rol la información que necesita para su gestión.

Finalmente, las pruebas realizadas durante y después de cada iteración permitieron identificar y corregir inconsistencias en validaciones, permisos y cálculos, fortaleciendo la robustez final del sistema y su correspondencia con los requerimientos establecidos en la descripción del problema.